

부록 A. 파이참 통합 개발환경

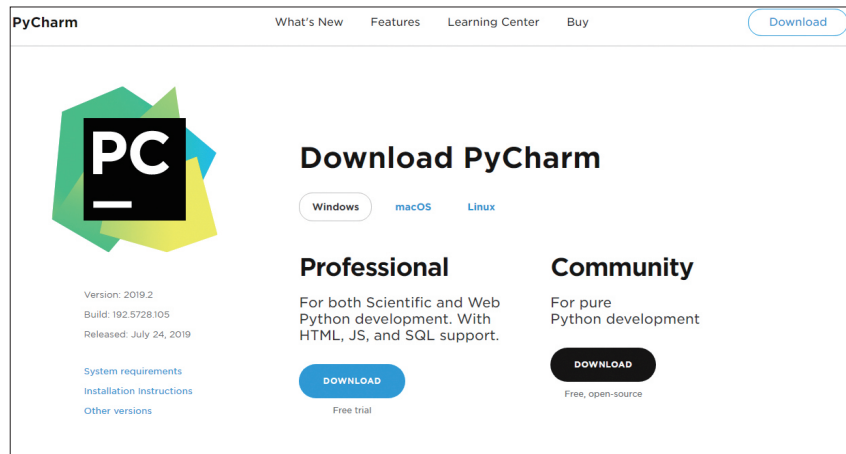
프로그램을 개발하기 위해서는 코딩과 디버깅, 번역, 배포 등의 많은 작업이 필요하다. 이러한 작업을 수행하는 하나의 프로그램을 **통합 개발 환경**Integrated Development Environment, IDE이라고 하는데, C/C++ 언어를 위해서는 Visual Studio, Xcode와 같은 통합 개발 환경 프로그램을 사용하며 Java 언어의 개발을 위해서는 Eclipse 등의 프로그램을 널리 사용한다.

파이썬 소프트웨어 재단에서는 **IDLE: Integrated Development and Learning Environment**라고 하는 프로그램을 제공하는데 이 프로그램은 기본 파이썬 통합 개발 환경 프로그램으로 널리 이용되고 있다. 이 밖의 통합 개발 환경으로는 **아톰**Atom과 **파이참**PyCharm, **스파이더**Spyder 등 여러 종류의 프로그램이 있으며 이 통합 개발환경은 에디터, 인터프리터와의 연동, 디버깅, 코딩 스타일 자동 수정 등 편리한 기능을 제공한다. 이 책에서는 모든 IDE를 다뤄 볼 수는 없기 때문에 파이참의 사용법을 통해 IDE의 사용방법을 알아보도록 하겠다.

PyCharm은 JetBrains사에서 개발한 파이썬 개발도구로 Professional 버전과 Community 버전의 두 가지가 있다. 이 중 Community 버전의 경우 오픈소스이며 아파치 2.0 라이선스로 무료 배포되고 있다. PyCharm은 다음 웹 사이트에서 다운받을 수 있다.

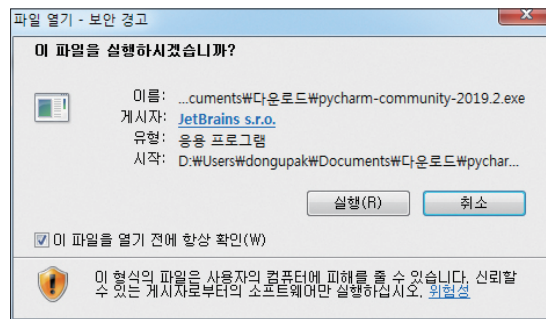
<https://www.jetbrains.com/pycharm/>

[그림 A-1]은 파이참을 다운 받을 수 있는 JetBrains사의 웹사이트로 왼쪽에는 파이참 Professional 버전, 오른쪽에는 Community 버전을 다운 받을 수 있는 버튼이 있다. 파이참 Professional 버전은 연간 사용료를 납부해야하는 유료 버전이다.



[그림 A-1] 파이참을 다운 받을 수 있는 JetBrains 사의 웹사이트

다음 [그림 A-2]는 파일 열기 창으로 DOWNLOAD 버튼을 클릭하여 다운받은 파이참 설치 파일을 실행할 때 나타나는 화면이다. 이 화면에서 실행(R) 버튼을 선택하면 파이참의 설치가 이루어진다. 이 화면은 여러분의 파이참 설치 버전이 다를 경우 약간의 차이가 있을 수 있다.



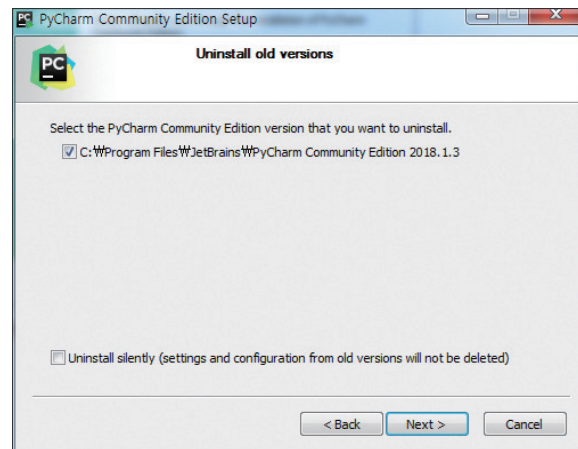
[그림 A-2] 파이참 설치 파일의 실행시 나타나는 화면

이어서 나타나는 화면은 다음 [그림 A-3]과 같으며 Next 버튼을 클릭하여 다음 화면으로 넘어 갈 수 있다.



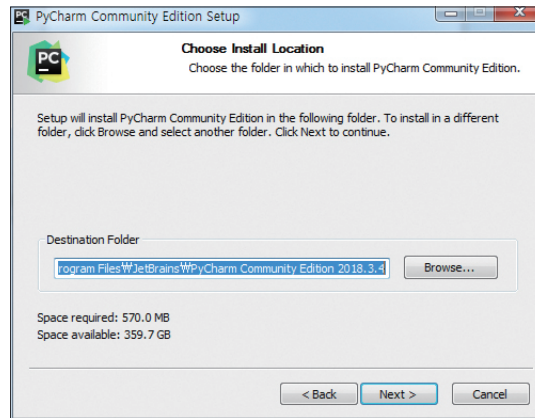
[그림 A-3] 파이참 설치시 나타나는 첫 화면

다음 화면은 [그림 A-4]와 같이 이전 버전의 파이참이 있을 경우 이를 삭제하겠다는 경고화면이다. 이 화면의 경우 2018년 1월 3일 버전의 파이참이 설치되어 있어서 이를 삭제하고 새 버전의 파이참을 설치하겠다는 의미이다.



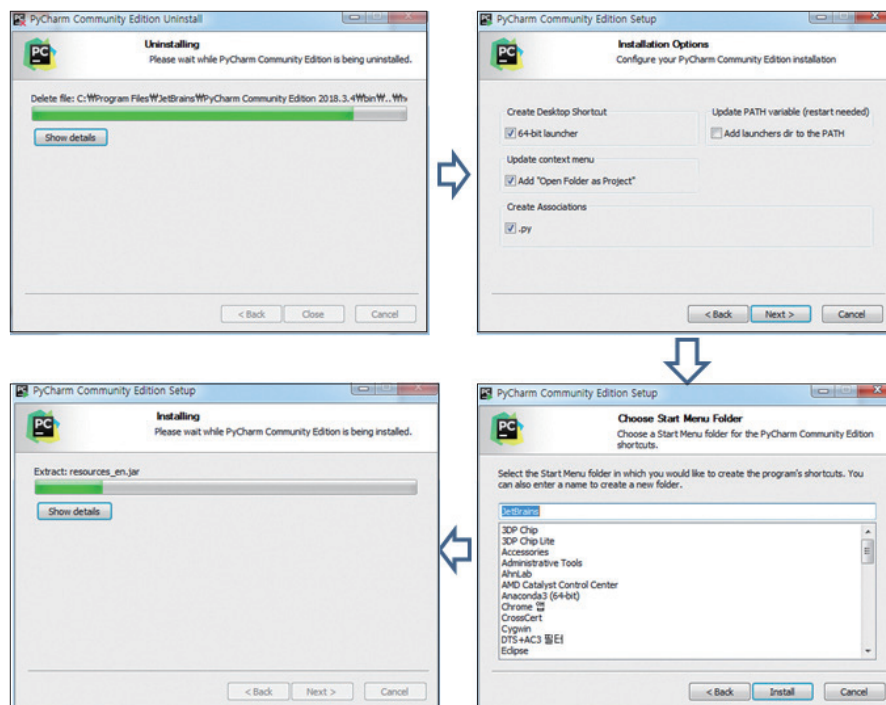
[그림 A-4] 파이참 설치 전 이전 버전이 있는 경우 삭제여부를 묻는 화면

기존의 파이참이 제거되면 다음 [그림 A-5]와 같이 설치 위치를 선택하는 화면이 나타나며 설치위치를 Browse... 버튼을 이용하여 다시 정의할 수 있다. 이 화면에서 Next를 선택하여 디폴트 경로에 설치되도록 하였다.



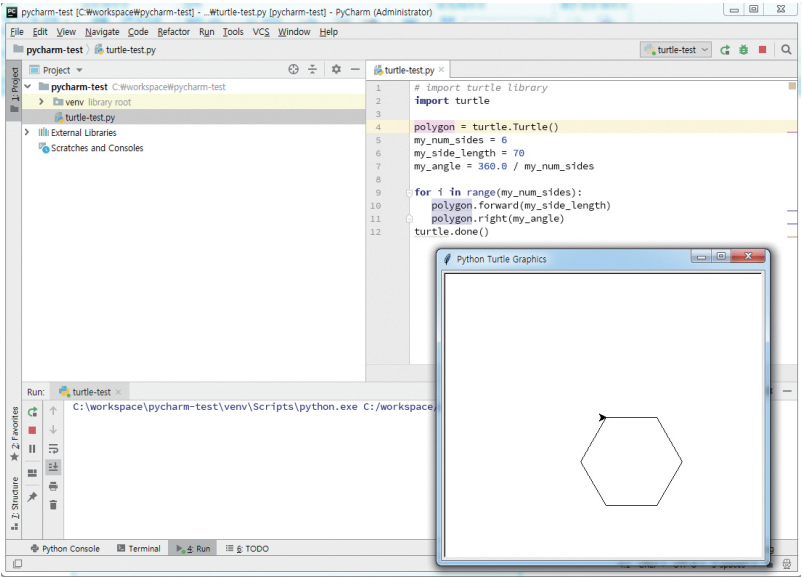
[그림 A-5] 파이참 설치 디렉토리

다음 단계로 설치 옵션(Installation Options)을 선택하는 메뉴인데 [그림 A-6]과 같이 데스크탑의 단축 구동기와 관련된 확장자(.py) 등을 그림과 같이 선택하고 Next를 선택하면 설치가 진행된다.



[그림 A-6] 파이참 설치 화면

이제 [그림 A-6]의 세 번째 항목과 같이 시작 메뉴의 어느 폴더 아래 파이참 실행아 이콘을 둘 것인가를 선택하고 설치를 진행하면 설치가 완료된다. [그림 A-7]은 파이참을 사용하여 터틀 그래픽 프로그램을 작성하고 실행한 결과이다.



[그림 A-7] 파이참을 이용한 터틀 그래픽 프로그램과 실행 창

 NOTE : 파이썬 통합 개발 환경의 순위

THE BEST 2 OF 41 OPTIONS 

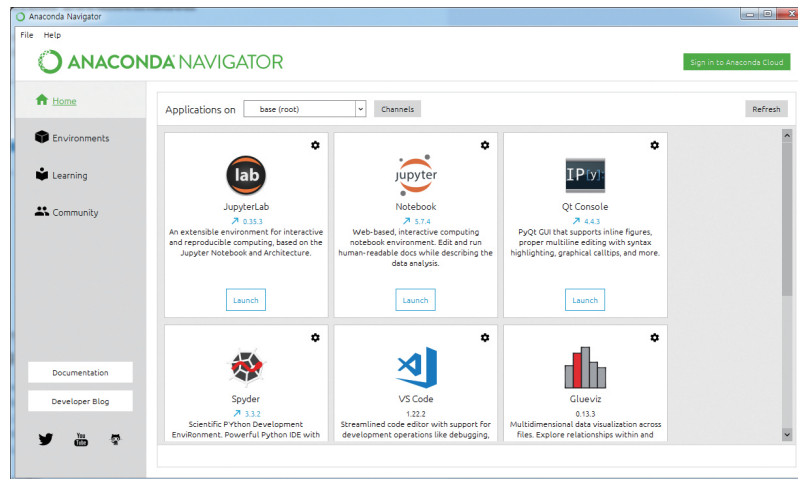
BEST PYTHON IDES OR EDITORS		PRICE	MULTI LANGUAGE SUPPORT	CROSS PLATFORM
91	 PyCharm Professional E...	£6.90 - £49.90 /MONTH	Yes	Windows, macOS, Linux, F
91	 Visual Studio Code	FREE	Yes	Yes
87	 Vim	FREE	Yes	Yes
85	 Spacemacs with Pytho...	-	Yes	Yes
84	 Sublime Text	\$80	Yes	Yes

[그림 A-8] slant.co 웹 사이트에서 공개한 파이썬 통합 개발환경의 순위

[그림 A-8]은 <https://www.slant.co/topics/366/~best-python-ides-or-editors> 웹 사이트에서 공개한 파이썬 통합개발 환경 도구(IDE)들의 순위를 나타내고 있다(2019년 7월의 순위임). 1위부터 차례대로 PyCharm, Visual Studio Code, Vim, Spacemacs with Python, Sublime Text 순으로 나열되어 있다. 물론 이 순위는 수시로 변동되는 순위이며, 여러분은 여러분의 개발목적에 가장 적합한 통합 개발 환경을 사용하면 된다.

부록 B. 아나콘다 패키지와 주피터 노트북

아나콘다Anaconda는 수학과 과학 분야에서 사용되는 여러 패키지들을 묶어 놓은 파이썬 배포판으로서 SciPy, Numpy, Matplotlib, Pandas, Tensorflow 등을 비롯한 많은 패키지들을 포함하고 있다. 아나콘다는 특히 최근에 데이터 사이언스와 머신 러닝 분야에서 파이썬을 사용하기 위해 사용하는 기본적인 배포판 역할을 하고 있다. 아나콘다를 설치하기 위해서는 <https://www.anaconda.com/> 웹사이트에서 자신의 OS에 맞는 프로그램을 다운받아 설치하면 된다. 통상 Python 3.x 버전을 선택한다. 아나콘다의 다운로드와 설치의 파이참과 마찬가지로 쉽게 할 수 있다. 다음 [그림 B-1]은 설치 후 **아나콘다 내비게이터**Anaconda Navigator를 실행시킨 화면이다.



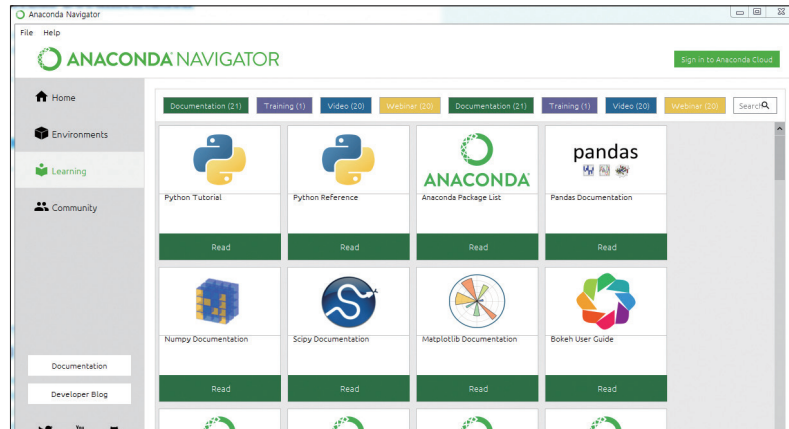
[그림 B-1] 아나콘다 내비게이터의 실행화면

이 내비게이터 화면의 첫 번째 항목인 JupyterLab은 주피터 프로젝트의 차세대 사용자 인터페이스로 웹 기반의 환경에서 파이썬 프로그램을 개발하고 실행할 수 있다. 다음으로 Jupyter Notebook이라는 웹 개발의 대화식 파이썬 개발환경인데 이 책에서는 **주피터 노트북**Jupyter Notebook의 사용법 위주로 설명할 것이다. 이밖에도 Qt Console, Spyder, VS Code 등의 여러 개발환경이 있다.

더불어 아나콘다 내비게이터의 왼쪽에 있는 Learning이라는 탭을 살펴보면 다음 [그림 B-2]와 같이 파이썬과 Pandas, Numpy, Scipy, Matplotlib 등의 라이브러리의 기능을

익힐 수 있는 상세한 튜토리얼을 제공한다. 이 튜토리얼은 가장 최신에 업데이트 된 내용을 충실하게 설명하고 있으니 잘 활용하면 많은 도움이 될 것이다.

자 이제 주피터 노트북의 사용법을 상세히 알아보도록 하자.

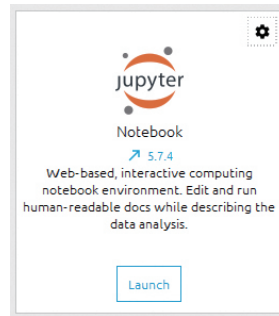


[그림 B-2] 아나콘다 내비게이터의 Learning 탭 화면과 튜토리얼

주피터 노트북은 여러 종류의 프로그램 코드를 웹 브라우저에서 실행하고 결과를 보여주는 대화식 개발환경이다. 이런 방식은 코드의 입력 즉시 그 결과를 확인할 수 있어서 프로그램을 처음 접하는 개발자나 데이터 분석을 하는 사람들에게 매우 적합한 개발환경이다. 주피터 노트북은 파이썬이라는 프로그래밍 언어만을 위한 것이 아니며, R 언어와 같은 통계처리에 널리 사용되는 언어나 Julia, Scala, Perl, PHP 등의 언어도 지원한다.

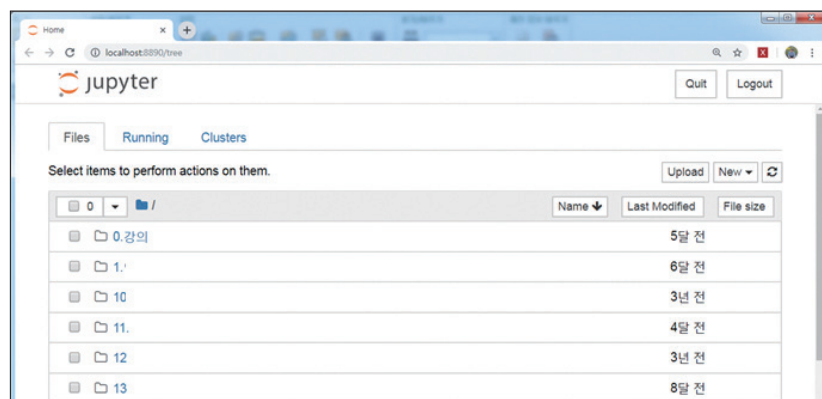
주피터 노트북

주피터 노트북은 개발환경이므로 일단 하나하나 수행해 보고 코드를 입력해 보면서 그 사용법을 익히는 것이 빠르다. 일단 아나콘다 내비게이터에서 수행하기 위해서는 [그림 B-3]과 같이 Jupyter Notebook의 Launch(실행) 버튼을 클릭하도록 하자.



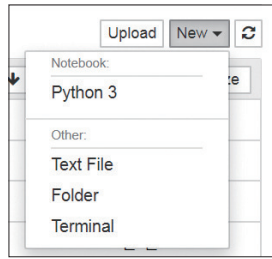
[그림 B-3] 아나콘다 내비게이터에 있는 주피터 노트북의 실행을 위한 메뉴

주피터 노트북을 실행하면 여러분의 컴퓨터에 있는 기본 웹 브라우저가 실행되며 이 웹 브라우저에서 기본 문서 폴더의 내용이 [그림 B-4]와 같이 나타날 것이다. 이 그림의 기본 문서 폴더에서 임의의 폴더를 클릭하여 이동할 수 있다.



[그림 B-4] 주피터 노트북의 실행 화면

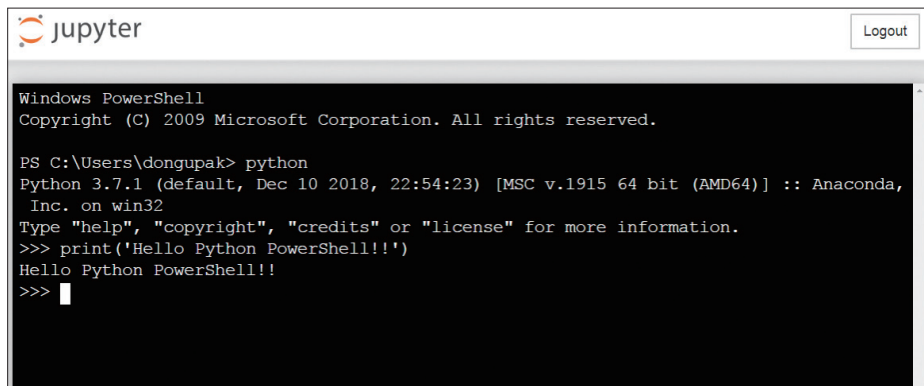
이제 우측 상단의 New 버튼을 선택하여 보자. 그러면 [그림 B-5]와 같이 하위 메뉴가 나타나는 것을 볼 수 있다. 그림의 첫 메뉴는 Python 3이라는 메뉴로 새로운 파이썬 3 버전의 주피터 노트북을 만들 수 있으며, Text File은 새 텍스트 파일을 생성하는 메뉴이다. 다음으로 Folder 메뉴는 새 폴더를 [그림 B-6]과 같이 생성할 수 있으며, Terminal 메뉴는 웹 브라우저상에서 [그림 B-7]과 같이 윈도 파워셸을 수행하는 기능이 있다. 이 파워셸 환경에서 여러분은 그림과 같이 파이썬을 수행하는 대화창을 열 수도 있다.



[그림 B-5] 주피터 노트북의 New 메뉴

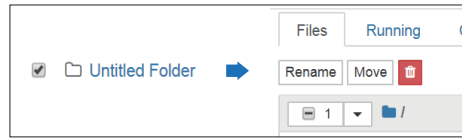


[그림 B-6] 주피터 노트북의 Folder 메뉴를 선택하여 생성한 새 폴더



[그림 B-7] 주피터 노트북의 Terminal 메뉴를 선택하여 생성한 윈도 파워셸 환경

앞서 생성한 "Untitled Folder"라는 폴더의 이름 앞에 있는 체크 상자 기호를 선택하면 [그림 B-8]과 같이 상단의 Files 메뉴가 Rename, Move, 삭제를 의미하는 쓰레기통 아이콘으로 나타나며 Rename을 선택하여 여러분이 작성하고자 하는 폴더 명으로 변경할 수 있다. 여기서는 [그림 B-9]와 같이 jupyter-work이라는 이름으로 변경하고 Rename 버튼을 클릭한 후 이 폴더 아래에서 작업을 해 보도록 하자.

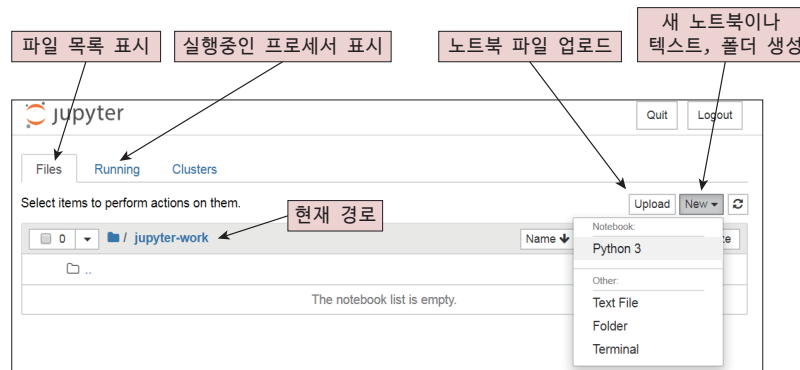


[그림 B-8] 폴더 이름을 변경하는 방법



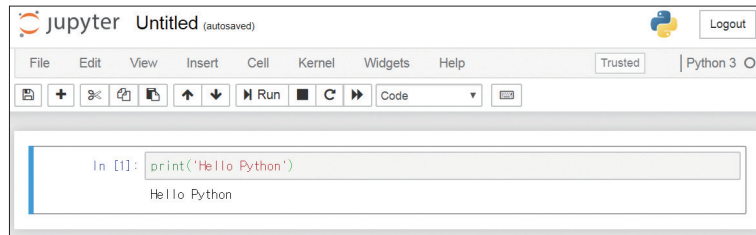
[그림 B-9] 주피터 노트북에서 디렉토리 이름을 "jupyter-work"으로 변경하는 방법

주피터 노트북에서 `jupyter-work`이라는 폴더로 이동을 하게 되면 브라우저의 상단에 `jupyter-work`이라는 이름의 현재 폴더 이름이 나타난다. [그림 B-10]과 같이 File 탭은 파일의 목록을 표시하는 기능이 있으며, Running 탭에는 현재 실행중인 프로세스를 표시한다. 그리고 아래의 `/jupyter-work`은 현재의 경로를 표시하는데 이 표시에 나타나는 폴더를 선택하여 상위 폴더로 이동할 수 있다. 상위 폴더의 이름은 `..`으로 표시된다. 그리고 상단의 Upload 버튼은 노트북 파일을 업로드 하는 기능이 있으며, New 버튼은 새 노트북 파일이나 텍스트 파일, 폴더를 생성하는 기능이 있다. 이제 이 폴더에서 [그림 B-10]과 같이 New 메뉴의 Python 3을 선택해 보도록 하자.



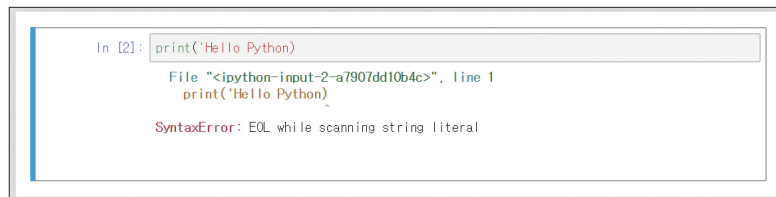
[그림 B-10] 주피터 노트북에서 새로운 파이썬 노트북 파일을 생성하기

이제 [그림 B-11]과 같이 Untitled라는 이름의 제목과 코드 입력창이 나타나며 이 코드 입력창에 `print('Hello Python')`과 같은 파이썬 코드를 입력하면 된다. 이제 이 코드를 실행하기 위해서는 1) Cell 메뉴의 Run Cells를 선택하거나, 2) Run이라는 도구 상자를 선택하거나, 3) 단축키 `Control+Enter`를 입력하는 방법을 사용할 수 있다. 이제 세 가지 방법 중 하나를 선택하여 이 코드를 실행하면 [그림 B-12]와 같이 Hello Python이 수행되어 그 결과가 파이썬 코드의 아래에 나타나는 것을 확인 할 수 있다.



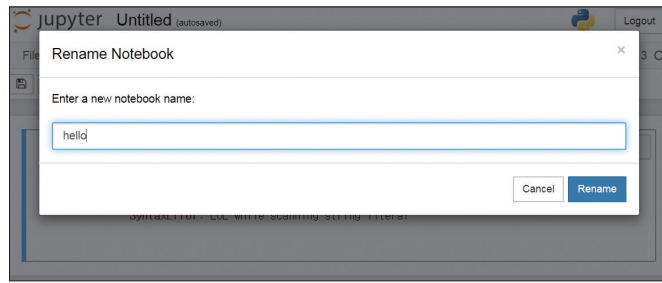
[그림 B-11] 주피터 노트북에서 파이썬 코드 입력하여 실행시키기

만일 입력된 코드에 오류가 있을 경우 [그림 B-12]와 같이 오류가 표시되며 오류를 수정하여 다시 실행하면 된다.



[그림 B-12] 주피터 노트북에서 입력한 파이썬 코드의 에러 표시

주피터 노트북에서 작성한 코드는 Untitled.ipynb라는 이름으로 자동 저장되는데 이 파일 이름은 상단의 Untitled(autosaved) 레이블을 클릭하여 변경할 수 있다. [그림 B-13]과 같이 hello라는 이름으로 새 노트북 이름을 입력하면 지정된 폴더에 hello.ipynb라는 이름의 파일이 만들어진다. ipynb는 주피터 노트북 파일의 확장자인데 IPython notebook의 약자이다. IPython은 대화식 파이썬 개발을 위한 프로젝트로 파이썬 언어를 사용하여 다양한 환경에서 파이썬을 실행시키고자하는 프로젝트이다.



[그림 B-13] 주피터 노트북 파일의 이름 변경

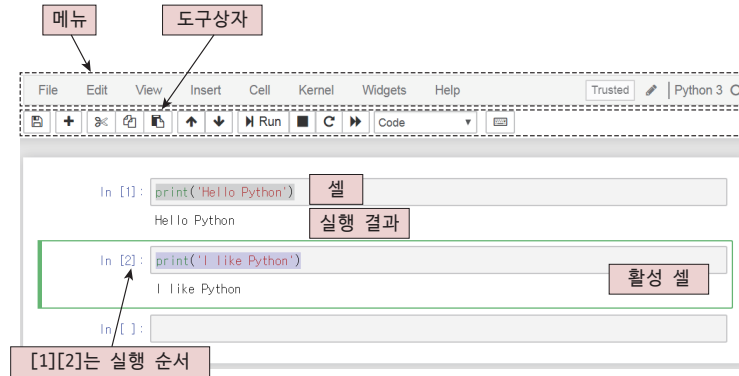
주피터 노트북의 상세기능

주피터 노트북을 이용하여 파이썬 코드를 입력하고 실행시키고 저장하는 기능을 완료하였다면 이제 주피터 노트북의 기능을 좀 더 상세하게 알아보도록 하자. 우선 셀의 추가와 삽입, 그리고 이동에 대하여 우선 알아보도록 하자.

주피터 노트북에서 명령어를 입력하여 실행할 수 있는 입력 공간을 **셀cell**이라고 하는데 모든 명령은 셀 단위로 입력되며 셀을 추가하거나 이동시켜서 명령어를 추가하고 순서를 변경시킬 수 있다. 우선 다음 [그림 B-14]와 같이 `print('Hello Python')`가 입력된 셀 다음의 도구상자에 있는 + 버튼을 추가하여 `print('I like Python')`을 입력한 후 다시 한번 실행해 보도록 하자. 이제 [그림 B-14]와 같은 화면을 볼 수 있는데 이 그림을 통해 각 구성 요소별 명칭과 역할에 대해서 알아보도록 하자. 우선 그림 상단의 메뉴에서 File 메뉴는 주피터 노트북의 파일을 만들거나 저장하거나 다운로드 받을 수 있는 기능이 있으며, Edit 메뉴는 주로 셀을 만들거나 이동하거나 삭제, 합병하는 기능을 가지고 있다. View 메뉴는 **도구상자toolbar**나 **헤더header**를 보여주거나 숨기는 기능을 Insert 메뉴는 셀을 현재 셀의 위나 아래에 추가하는 기능이 있다. Kernel 메뉴는 주피터 노트북의 파이썬 코드를 실행하거나 중지하는 기능이 있으며, Widgets 메뉴는 노트북 화면에서 만든 그래픽 사용자 요소를 저장하거나 삭제하는 기능이 있다. 제일 마지막에 나타나는 Help 메뉴를 통해서 여러분은 주피터 노트북의 도움말이나 키보드 단축키 등에 대한 상세한 정보를 얻을 수 있다.

도구상자는 노트북 파일의 저장, 새로운 셀 만들기, 셀 자르기, 복사, 붙여넣기, 이동, 실행, 정지, 재실행 등의 기능으로 이루어져 있다. 도구상자 아래에 있는 네모 블록을 셀이라고 하며 셀을 실행하면 다음 칸에 실행 결과가 나타나서 셀의 수행과정을 하나하나 살펴볼 수 있다. 키보드 화살표 키를 이용하여 셀과 셀 사이를 이동할 수 있는데 이때 실행 가능한 셀을 활성 셀 혹은 포커스 셀이라고 한다.

셀이 실행되면 In [1], In [2]와 같이 실행 순서가 표시되는데 전체 셀에서 어떤 순서로 셀이 실행되는가를 나타내는 중요한 표시이다.



[그림 B-14] 주피터 노트북 화면의 각 구성요소별 명칭

이제 [그림 B-15]와 같이 `num = 100`이라는 코드를 가지는 셀과 `num = num + 100`이라는 코드와 `print(num)`을 함께 포함한 셀을 만들어 보자. 두 셀은 [3], [4]와 같이 나타나며 수행결과 화면과 같이 `num` 변수에는 원래의 100에 100을 더한 200이 출력되는 것을 볼 수 있다. 이제 아래와 같이 [4]번 셀을 한번 더 수행해 보자 그러면 셀의 수행 번호는 [5]로 변경되는데 이때 [4]번 셀의 수행으로 인해 200으로 증가한 `num` 변수에 새로 100이 더해져서 그 결과는 300이 되는 것을 확인해 볼 수 있다.

이와 같이 셀의 수행 번호는 코드의 배열 순서와 상관없이 대괄호에 나타난 번호 순서와 밀접한 관계를 가진다는 것에 유의하도록 하자.



[그림 B-15] [3], [4] 셀에서 [4] 셀을 한번 더 수행한 후의 결과

그렇다면 어떤 원리에 의해서 웹 브라우저의 셀에 입력한 코드가 브라우저 위에서 실행 되는 것일까? 이 부분에 대해서 자세히 알아보도록 하자.



NOTE : 주피터 노트북 셀 실행 단축키

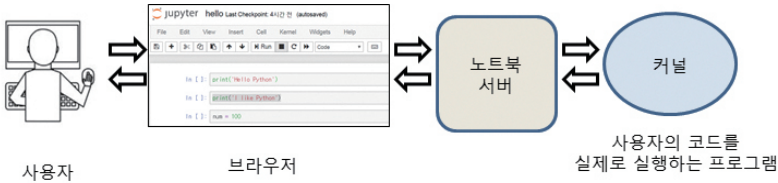
주피터 노트북의 셀을 실행할 적에는 메뉴의 Cell의 하위 메뉴인 Run Cells를 실행하는 방법도 있고 도구상자의 Run 버튼을 입력하는 방법도 있다. 하지만 키보드 입력시에 마우스와 키보드를 오가며 선택하는 것 보다는 키보드 단축 키를 이용하는 것이 더 편리하다. 셀 실행을 위한 단축키는 다음과 같은 것이 있다.

[표 B-1] 주피터 노트북 셀의 실행 단축키와 역할

단축키	역할
Control + Enter	셀 실행하기, 실행 후 활성 셀이 변하지 않는다.
Shift + Enter	셀 실행하기, 실행 후 활성 셀이 한 칸 아래로 이동한다.
Alt + Enter	셀 실행하기, 실행 후 현재 셀 아래에 새로운 셀을 삽입한다.

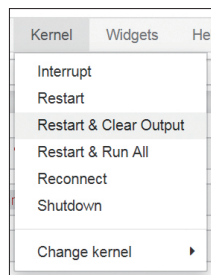
주피터 노트북과 커널

주피터 노트북의 구성요소를 나타내는 그림이 [그림 B-16]에 나타나 있다. 우리는 주피터 노트북 이용하여 프로그램을 작성하기 위해 웹 브라우저를 사용하였다. 이때 컴퓨터는 사용자의 입력을 웹 환경에서 처리할 수 있는 서버를 구동시킨다. 이 서버가 바로 **노트북 서버** **notebook server**이며 사용자의 입력이 들어오면 노트북 서버를 통해서 커널을 구동시킨다. **커널** **kernel**은 사용자가 입력한 코드를 실제로 실행하는 프로그램을 말하며 커널에서 구동된 실행 결과는 노트북 서버를 통해서 브라우저에 표시된다.



[그림 B-16] 사용자와 브라우저, 노트북 서버와 커널과의 관계도

다시 한번 커널의 메뉴를 살펴보면 첫 항목은 **Interrupt**로 현재 커널이 실행하고 있는 코드를 중지하는 명령이다. 현재 실행중인 코드는 셀의 왼쪽에 [*]와 같이 나타나며 이 코드가 실행이 완료되지 않은 상태에서 다음 셀로 이동하게 되면 실행은 되지만 순차적인 코드의 경우 오류가 발생할 수도 있다. Restart 메뉴는 커널을 재시작하게 된다. 따라서 코드상의 수행결과는 이전결과 그대로 남아 있지만 실제로 변수들이 가진 값들은 모두 초기화 된다. Kernel 메뉴의 Restart & Clear Output 메뉴를 선택하게 되면 커널이 다시 구동되면서 기존의 수행했던 결과물을 모두 소거하게 된다. 따라서 [그림 B-18]과 같이 결과물은 사라지고 실행 순서를 나타내는 []에는 빈 값이 표시되게 된다.



[그림 B-17] Kernel 메뉴의 하위 메뉴

```
In [ ]: print('Hello Python')

In [ ]: print('I like Python')

In [ ]: num = 100

In [ ]: num = num + 100
        print(num)
```

[그림 B-18] Kernel이 다시 구동된 후(Restart & Clear Output)의 노트북 화면

다음으로 나타난 Restart & Run All 코드는 다시 구동된 후 모든 코드를 순서대로 실행하는 유용한 기능이다. 따라서 이 메뉴를 실행하면 [그림 B-19]와 같이 셀이 나타나는 순서대로 [1] [2] [3] [4]와 같은 셀 수행 번호가 나타나는 것을 확인할 수 있다.


```
In [1]: print('Hello Python')
Hello Python

In [2]: print('I like Python')
I like Python

In [3]: num = 100

In [4]: num = num + 100
print(num)
200
```

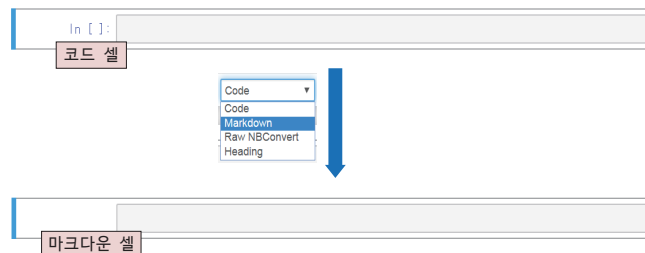
[그림 B-19] 커널이 다시 구동된 후 모든 코드를 실행한(Restart & Run All) 노트북 화면

다음 메뉴 Reconnect는 커널과의 연결이 끊어졌을 경우 다시 연결하는 기능이며, Shutdown 메뉴는 커널과의 연결을 완전히 종료하는 기능이다. 만일 Python 3 이외의 커널이 있을 경우 Change kernel 메뉴에서 다른 커널을 선택하는 것도 가능하다.

주피터 노트북에서 마크다운 기능 사용하기

주피터 노트북이 쓸모있는 이유 중 하나는 **마크다운** markdown이라는 기능을 통해서 코드 뿐만 아니라 읽기 쉬운 문서를 작성하는데도 유용하기 때문이다. 마크다운이란 텍스트에 기반한 마크업 언어로 쉽게 쓰고 읽을 수 있으며 매우 간단한 문법을 가지고 있어서 웹 상에서 문서를 빠르게 생성하고 직관적으로 관리를 할 수 있다.

역시 간단한 예제를 통해서 마크다운에 대해서 알아보도록 하자. 우선 [그림 B-20]과 같이 빈 셀을 하나 만들어서 Code라는 이름의 콤보박스 메뉴에서 Markdown이라는 메뉴를 선택해 보도록 하자. 이제 기본 셀인 코드 셀은 마크다운 셀로 변경되었는데 이 때문에 코드 셀의 실행 순서를 나타내는 In []: 이 사라진다.



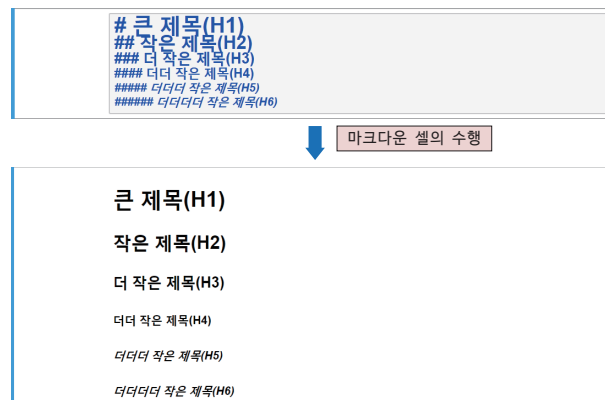
[그림 B-20] 코드 셀을 마크다운 셀로 변경하는 방법으로 코드 셀의 [] 표시가 사라짐

마크다운 셀에서 입력한 텍스트와 코드 명령어는 커널에서 실행되지 않으며, 코드 셀에서 입력한 명령만 커널에서 실행된다는 것을 한번 더 명심하도록 하자. 이때 두 셀을 구분하는 방법은 실행 순서가 나타난 셀인지 아닌지를 살펴보면 된다.

이제 마크다운 셀에 다음과 같은 간단한 텍스트를 입력하고 그 수행 결과를 확인해 보도록 하자.

```
# 큰 제목(H1)
## 작은 제목(H2)
### 더 작은 제목(H3)
#### 더더 작은 제목(H4)
##### 더더더 작은 제목(H5)
##### 더더더더 작은 제목(H6)
```

[그림 B-21]과 같이 마크다운 셀에서 입력한 내용의 #로 시작하는 부분은 글씨의 크기를 나타내는 명령어로 처리되어 나타나며 #의 개수가 1개에서 6개로 변할 때마다 제목의 크기가 작아지는 것을 볼 수 있다. 마크다운 셀에서 #는 헤더를 나타내는 명령어로 최대 6개까지 사용가능하다.



[그림 B-21] 마크다운 셀의 입력과 수행결과

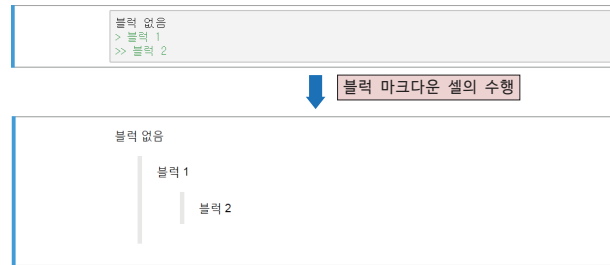
다음은 블록 인용구를 사용하는 방법이다. 블록 인용구의 사용법을 알아보기 위해 다음과 같은 텍스트를 마크다운 셀에서 입력해 보도록 하자.

블록 없음

> 블록 1

>> 블록 2

마크다운 셀에서 아무런 마크를 하지 않고 "블록 없음"과 같은 텍스트를 입력하게 되면 아래 [그림 B-22]와 같이 입력한 텍스트가 그대로 나타난다. 하지만 "> 블록 1"과 같이 > 마크를 한 후 문자를 입력하면 문자에 대한 들여쓰기가 이루어진다. 만일 ">> 블록 2"와 같이 연속된 > 마크를 사용하게 된다면 한 칸 더 들여쓰기가 이루어진 블록이 나타나게 될 것이다.



[그림 B-22] 마크다운 셀에서 블록을 이용한 들여쓰기.

이외에도 다음과 같은 유용한 마크다운 기능이 있으니 참고하여 하나하나 실행시켜 보도록 하자.

[표 B-2] 마크다운 기호와 하는 일

마크다운 기호	하는 일
1. 2. A.	순서 있는 목록을 표시한다. 이때 1. 2. 3.으로 하던 1. 1. 1.로 하던 표시되는 순서는 모두 1. 2. 3.으로 동일하다. 들여쓰기를 이용하여 하위 항목을 생성하는 것도 가능하다.
* * *	글머리 기호를 표시한다. 들여쓰기가 가능하다.
+ + +	글머리 기호를 표시한다. 들여쓰기가 가능하며, 위의 기호와 동일하다.

<pre>- - - <pre> ~ </pre> 혹은 ... ~ ... ** ~ ** 혹은 -- ~ -- * ~ * 혹은 _ ~ _</pre>	글머리 기호를 표시한다. 들여쓰기가 가능하며, 위의 기호와 동일하다. ~ 있는 부분을 있는 그대로 화면에 표시한다. 예들 들어 순서 있는 목록이나, 글머리 기호를 화면에 나타내는데 사용할 수 있다. ~ 있는 부분을 볼드체 글꼴로 출력한다. ~ 있는 부분을 이탤릭체 글꼴로 출력한다.
--	---



LAB B-1 : 마크다운 셀의 순서 기호, 글머리 기호 등의 사용법

1. 마크다운 셀의 마크다운 기호를 사용하여 다음과 같은 출력이 나타나도록 하여라.

(a)

- ```
1. 첫번째 목록
2. 두번째 목록
3. 세번째 목록
```

(b)

- ```
• 글머리 기호 1
  ▪ 글머리 기호 2
    ◦ 글머리 기호 3
```

(c)

```
# 헤드라인 표시는 해시마크를 사용합니다
그리고 글머리 기호는 다음과 같이 표시합니다.
* 글머리 기호 1
* 글머리 기호 2
* 글머리 기호 3
```

(d)

```
아름다운 대한민국 삼천리 금수강산
```

다음은 다른 웹 사이트나 자료의 위치에 대한 [하이퍼링크](#) hyperlink를 설정하는 방법인데 다음과 같은 문법에 따라 설정할 수 있다.

[텍스트](텍스트에 대한 하이퍼 링크)

이제 마크다운 셀에서 다음과 같은 내용을 입력해 보도록 하자. 그리고 이 마크다운 셀을 수행하면 [그림 B-23]과 같은 내용이 나타나는 것을 볼 수 있다.

[생능출판사](https://www.booksr.co.kr/)

생능출판사

[그림 B-23] 하이퍼링크의 화면 출력 결과

이 하이퍼링크에서 대괄호([])를 생략하게 되면 다음과 같이 텍스트와 하이퍼링크가 동시에 나타나게 된다.

생능출판사(https://www.booksr.co.kr/)

[그림 B-24] 하이퍼링크의 다른 출력 결과

만일 현재 디렉토리 아래에 images라는 폴더에 logo.gif라는 이미지 파일이 있을 경우 다음과 같은 경로를 설정하여 이미지 파일을 가져올 수 있는데 이는 HTML 파일의 이미지를 가져오는 방법과 동일하다.

```
<img src='../images/logo.gif'>
```

혹은 다음과 같은 인터넷 주소를 이용하여 출판사의 로고를 삽입할 수도 있다.

```
<img src='https://www.booksr.co.kr/images/common/logo.jpg'>
```

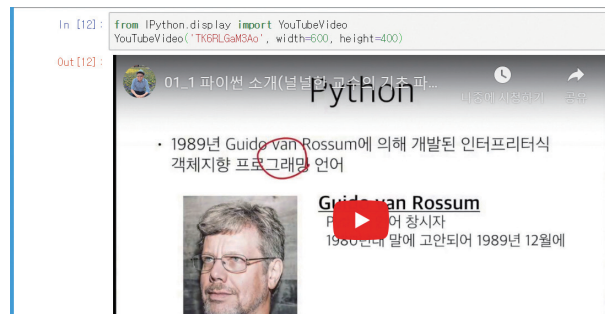


[그림 B-25] 인터넷 주소를 이용하여 출판사의 로고를 표시한 결과

NOTE : 주피터 노트북에서 유튜브 동영상 삽입하기

주피터 노트북에서 자신의 코드를 보여주고 이 코드의 수행결과를 유튜브 동영상으로 보여주는 방법은 매우 자주 이용되는 유용한 기능이다. 이때는 IPython 모듈의 YouTubeVideo() 함수를 사용하면 된다. 다음은 이때 사용되는 코드의 예이다. 이 코드는 [그림 B-26]과 같이 코드 셀에서 실행하는 파이썬 명령어이다. 여기에서 사용된 'TK6RLGaM3Ao' 코드는 유튜브 동영상의 주소창에 나타나는 동영상의 아이디이다. 이 때 [그림 B-27]의 watch?v= 다음에 나타나는 숫자와 알파벳의 조합기호가 바로 유튜브 동영상의 고유한 아이디 값이다. 그리고 width와 height는 동영상의 너비와 높이이다.

```
from IPython.display import YouTubeVideo
YouTubeVideo('TK6RLGaM3Ao', width=600, height=400)
```



[그림 B-26] 코드 셀에서 파이썬 코드를 이용하여 유튜브 동영상을 삽입한 결과

<https://www.youtube.com/watch?v=TK6RLGaM3Ao>

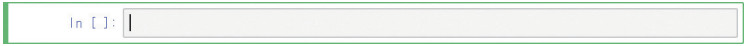
[그림 B-27] 유튜브 동영상의 아이디

이밖에도 마크다운 셀에서는 표 만들기 기능과 HTML 태그를 사용할 수 있는 다양한 방법이 존재한다.

주피터 노트북의 명령 모드

주피터 노트북의 특징 중 하나는 명령모드와 입력모드의 두 가지 모드가 존재한다는 것인데 이 모드에 대해서 상세하게 알아보도록 하자. 여러분이 주피터 노트북에 새로운 내용을 입력하기 위해서는 단순히 한 셀을 선택한 후 Code 혹은 Markdown을 선택하고 입력

을 하면 되는데 이때 입력 셀의 모습은 [그림 B-28]과 같은 녹색 테두리 선에 커서가 깜박인다.

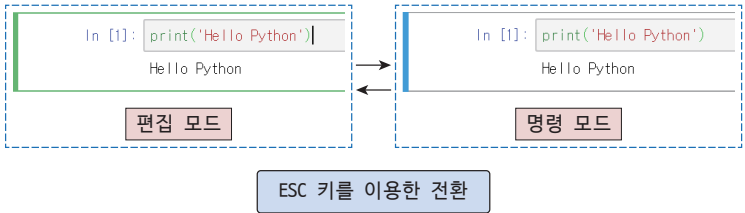


[그림 B-28] 코드 셀의 입력 전 모습(편집 모드)



[그림 B-29] 코드 셀의 입력과 코드 실행 후의 모습(명령 모드)

이 셀에 명령을 입력한 후 실행을 하게 되면 [그림 B-29]와 같이 셀의 테두리 색상이 하늘색으로 변하고 코드가 수행되며 그 결과가 나타난다. 이 때 녹색 테두리 선이 나타나는 경우를 편집 모드edit mode라고 하며 하늘색 테두리 셀이 나타나는 경우를 명령 모드command mode라고 한다. 명령 모드에 있을 경우 키보드 단축 키를 사용하여 셀을 복사하거나 화살표 키를 사용하여 셀 간을 이동하는 작업을 편리하게 할 수 있으며 두 모드간의 전환은 [그림 B-30]과 같이 ESC 키를 이용하여 할 수 있다.



[그림 B-30] 셀의 편집 모드, 명령 모드

명령 모드에서 사용가능한 단축키들은 다음 [표 B-3]과 같은 것들이 있으며 이외에도 수 십 가지 이상의 단축키들을 유용하게 사용할 수 있다.

[표 B-3] 명령 모드의 단축 키와 하는 일

명령 모드 단축키	하는 일
UP/DOWN 키 k, j 키	셀 간의 위(UP), 아래(DOWN) 이동 기능을 한다. 이때 k 키는 UP 키와 동일하며, j 키는 DOWN 키와 동일하다.
s	노트북 파일을 저장한다.
y	코드 셀 유형으로 변환시킨다.
m	마크다운 셀 유형으로 변환시킨다.
1, 2, 3, 4, 5, 6	큰 제목 H1부터 작은 제목 H6까지 셀을 수정한다.
a	현재 셀 위에(above) 새로운 셀을 만든다.
b	현재 셀 아래에(below) 새로운 셀을 만든다.
d + d	d를 2회 연속 입력하여 현재 셀을 삭제한다.
z	셀 삭제 명령을 취소한다.
c	현재 선택된 셀을 복사한다.
v	복사된 셀을 현재 선택된 셀 아래에 붙여 넣는다.
Shift + UP/DOWN	여러 셀을 동시에 선택한다.
Shift + m	선택된 여러 셀을 합병(merge)하여 하나의 셀로 만든다.

부록 C. 클라우드 환경의 파이썬 개발

구글 colab 환경에서 파이썬 개발하기

파이썬을 이용하는 여러가지 방법이 있으나 그 중에서도 구글의 **Colaboratory**는 클라우드 환경에서 주피터 노트북 기반의 파이썬 개발을 위한 탁월한 환경을 제공한다. 줄여서 **colab**이라고 부르는 이 웹 서비스는 주피터 노트북을 구글의 서버에서 구동시켜 사용자가 이 서버에 접속하는 방식으로 파이썬을 이용할 수 있다.

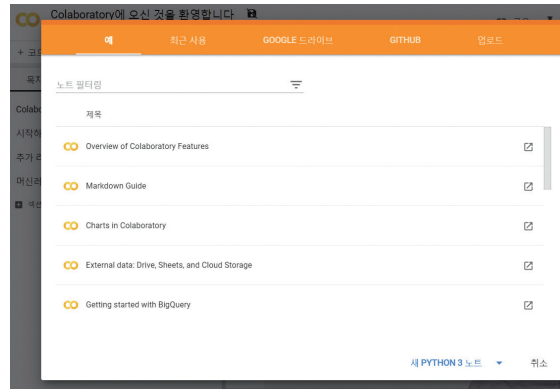
이 서비스의 장점을 설명하면 크게 다음과 같은 것들이 있다.

1. 파이썬을 설치하지 않고도 웹 브라우저를 이용하여 주피터 노트북에서 파이썬을 사용할 수 있다.
2. 다른 사용자들과의 파일 공유가 가능하며 협업을 통한 개발도 손쉽게 할 수 있다.
3. numpy, pandas, scikit-learn, tensorflow, pytorch 등과 같은 데이터 분석 및 머신러닝 패키지들이 설치되어 있어서 개별적으로 설치할 필요가 없다.
4. 클라우드에서 제공하는 **GPU(그래픽스 처리 장치Graphics Processing Unit)**와 **TPU(텐서 처리 장치Tensor Processing Unit)**를 사용할 수 있다.
5. 구글 드라이브의 문서 생성과 공유가 가능하다.

구글 colab의 정식 명칭은 Google Colaboratory로 구글 드라이브와 주피터 노트북 환경에서 협업을 통한 개발이 가능하다. 접속을 위한 웹 사이트는 다음과 같으며 사용을 위해서는 **구글 계정**이 필요하다.

<https://colab.research.google.com/>

자 이제 준비가 되었다면 이 웹 사이트에 접속을 하도록 하자. 첫 접속이 되면 다음 [그림 C-1]과 같은 메뉴가 나타날 것이다. 각 메뉴는 colab을 소개하는 내용과 마크다운 사용법 노트북에서 차트 만드는 방법, 외부 데이터의 사용법을 자세히 안내하는 좋은 내용으로 되어 있다. 우리는 제일 아래쪽의 "새 PYTHON 3 노트" 메뉴를 선택하여 새 파이썬 노트북을 생성하도록 하자.



[그림 C-1] colab 환경에 접속할 때 나타나는 메뉴

새로운 파이썬 노트북을 생성하게 되면 [그림 C-2]와 같은 익숙한 노트북 화면이 나타나며 간단한 코드를 입력하는 것으로 파이썬을 실행할 수 있다. 작성된 코드는 Untitled.ipynb 파일과 같이 디폴트 파일 이름으로 저장되는데 노트북에서 이미 작업한 바와 같이 파일 이름을 선택하여 수정할 수 있다. 코랩 환경에서 코드를 입력하거나 텍스트를 입력하여 마크다운 문자를 사용하는 방법은 PC 환경에서 주피터 노트북을 이용하는 방법과 매우 유사하다.



[그림 C-2] 주피터 노트북 환경을 웹 상에서 사용할 수 있는 colab 개발 환경

새로운 노트북 환경에서 상단의 "+ 코드", "+ 텍스트" 메뉴를 입력하여 코드와 텍스트를 생성한 후 입력할 수 있으며 셀 선택 후에 런타임 메뉴의 실행 모드를 선택하여 파이썬 코드를 실행할 수 있다. 코랩은 주피터 노트북의 단축키와 100% 호환되지 않으며 구글 드라이브와의 연동기능, 그리고 노트북 파일의 공유기능 등이 있다.

주피터 노트북은 가상의 서버에서 실행되고 있는데 이 서버의 사양은 [그림 C-3]과 같은 유닉스 운영체제 명령어를 입력하여 살펴 볼 수 있다. 우선 `!(느낌표)` 명령을 사용하여

커널에서 외부 명령어를 이용할 수 있는데 `!cat /etc/issue.net` 명령으로 현재 운영체제를 확인 할 수 있다. 여기서 살펴본 바와 같이 Ubuntu라는 리눅스 운영체제를 사용하며 운영체제의 버전은 18.04.2임을 알 수 있다. CPU의 정보는 `!cat /proc/cpuinfo` 명령으로 살펴 볼 수 있는데, 그림과 같이 Intel Xeon CPU를 사용하는 서버에서 커널이 구동됨을 알 수 있다.

```
[2] 1 !cat /etc/issue.net

↳ Ubuntu 18.04.2 LTS

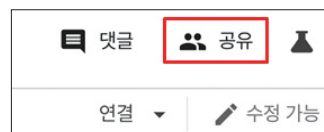
[3] 1 !cat /proc/cpuinfo

↳ processor      : 0
   vendor_id     : GenuineIntel
   cpu_family    : 6
   model         : 63
   model name    : Intel(R) Xeon(R) CPU @ 2.30GHz
   stepping      : 0
   microcode     : 0x1
   cpu MHz       : 2300.000
   cache size    : 46080 KB
   physical id   : 0
   siblings      : 2
   core id       : 0
   cpu cores     : 1
   apicid        : 0
   initial apicid : 0
   fpu           : yes
```

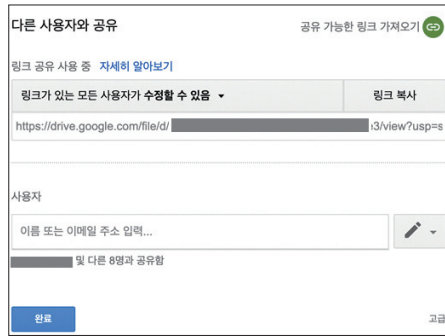
[그림 C-3] colab 환경의 운영체제와 CPU 정보

서버의 운영체제와 CPU, 메모리 사양 등 구동 환경은 구글의 정책과 상황에 따라서 변경될 수 있으므로 위의 그림과 동일하지 않을 수 있음을 명심하도록 하자. 그리고 유닉스 명령어를 이용한 시스템 설정을 알아보는 것 역시 여러분의 몫으로 남겨두도록 한다.

대부분의 기능은 주피터 노트북과 동일하지만 [그림 C-4]와 같이 우측 상단의 공유 버튼을 선택하면 [그림 C-5]와 같은 "다른 사용자와 공유" 창이 나타난다.



[그림 C-4] colab 환경의 공유 버튼



[그림 C-5] colab 환경에서 작성한 코드의 공유기능

[그림 C-5]의 코드 공유는 링크 공유를 통해서 다른 사람과 공유할 수 있으며, "아래의 이름 또는 이메일 주소 입력..." 기능을 이용하여 다른 사람을 초대할 수도 있다.

구글의 colab 이외에도 마이크로소프트에서 제공하는 노트북 환경도 있으며 다음의 주소로 접속할 수 있다. 이 서비스 역시 마이크로소프트사의 계정이 필요하다.

<https://notebooks.azure.com>



LAB C-1 : colab 코드 셀에서 유닉스 명령어 실행시키기

1. 유닉스 명령어 중에서 현재 컴퓨터의 디스크 용량을 확인하는 명령은 무엇인가?
(a) _____
2. 1번의 명령어를 사용하여 확인한 colab 서비스를 제공하는 시스템의 디스크 용량은 얼마인가?(단위 : 기가 바이트)
(a) _____
3. colab 환경에서 제공하는 파이썬의 버전을 확인하는 명령은 무엇인가? 그리고 본인의 시스템에서 확인한 버전은 얼마인가?

부록 D. 파이썬 출력서식

파이썬의 출력문과 % 서식

파이썬은 강력한 문자열 출력 서식을 제공하며, 이 서식을 이용하여 출력할 때 `format()` 메소드와 함께 % 서식을 이용할 수도 있다. 이 방법은 `format()` 메소드를 사용하는 것보다 간결한 표현이 가능하며, 문법적으로는 C 프로그래밍 언어의 출력 포매팅과 유사하다. 다음과 같은 실습을 통해 사용법을 알아보도록 하자.

대화창 실습 : `format()` 메소드와 % 서식의 사용방법

```
>>> print('{}, {}'.format('Hello', 'World!'))
Hello, World!
>>> print('%s, %s' % ('Hello', 'World!')) # format() 메소드 서식과 동일한 출력
Hello, World!
>>> print('{}, {}'.format(100, 200))
100, 200
>>> print('%d, %d' % (100, 200)) # format() 메소드 서식과 동일한 출력
100, 200
```

% 서식은 [그림 D-1]과 [그림 D-2]와 같이 % 다음에 %s를 통해 문자열을, %d를 통해 정수 값을 각각 출력할 수 있다.

`'%s, %s' % ('Hello', 'World!')`



[그림 D-1] 문자열 출력을 위한 %s 출력 지정자와 출력 인자

`'%d, %d' % (100, 200)`



[그림 D-2] 정수 출력을 위한 %d 출력 지정자와 출력 인자

뿐만 아니라, % 서식은 % 다음에 출력구간의 크기, 실수 자료 값의 경우 소수점 이하 자리수 등과 같은 상세한 정보를 다음과 같은 형식으로 지정하여 출력할 수 있다.

'%{N.M}자료형서식' % (자료값)

% 서식의 {N,M}은 생략 가능하지만 자료형 서식은 반드시 명시하여야 한다. 자료형은 s, d, o, x, f, e와 같은 알파벳 소문자로 서식 지정이 가능한데 다음 표의 %Ns와 같이 출력 칸의 수 N을 지정할 수 있다. 이때 N이 양수(+ 기호 사용)이면 오른쪽 정렬, 음수(- 기호 사용)이면 왼쪽 정렬을 하는데 이 기호는 생략할 수도 있다.

[표 D-1] 파이썬 출력문의 %서식에서 사용되는 포맷과 의미

포맷	의미
%{N}s	문자열 자료를 N칸에 맞게 문자열로 출력
%{N}d	int 형의 데이터를 N칸에 맞게 10진수로 출력
%{N}o	int 형의 데이터를 N칸에 맞게 8진수로 출력
%{N}x	int 형의 데이터를 N칸에 맞게 16진수로 출력

위의 포맷을 참고하여 다음과 같이 문자열을 출력해 보자.

대화창 실험 : % 서식을 이용한 문자열 출력과 좌우정렬

```
>>> print('%s, %s' % ('Hello', 'World!'))
Hello, World!
>>> print('%10s, %10s' % ('Hello', 'World!')) # 출력을 위해 10칸의 공간을 확보
Hello,      World!
>>> print('%-10s, %-10s' % ('Hello', 'World!')) # - 기호 사용시 왼쪽 정렬
Hello      , World!
```

문자열 %s를 사용할 경우 문자열 출력을 위한 칸의 크기는 출력 문자열의 길이에 따라 달라진다. 하지만 %10s와 같이 10을 입력하면 10칸의 문자열 출력 공간을 확보하고 이에 맞게 문자열을 출력하는 것을 확인할 수 있다. 여기서는 10칸의 출력 크기를 명시하였

으며 기본 정렬방식은 오른쪽 정렬임을 알 수 있다.

다음은 % 서식을 이용한 정수 출력의 예이다. 정수 출력은 10진수, 8진수, 16진수 출력이 모두 가능하다. 10진수 100은 8진수로 144, 16진수로 64임을 다음 출력을 통해 확인할 수 있다.

'''- 대화창 실험 : % 서식을 이용한 10진수, 8진수, 16진수 출력

```
>>> print('%d, %d' % (100, 200))
100, 200
>>> print('%10d, %10d' % (100, 200)) # 출력을 위해 10칸의 공간을 확보
      100,         200
>>> print('%10o, %10o' % (100, 200)) # 8진수로 출력
      144,         310
>>> print('%10x, %10x' % (100, 200)) # 16진수 형태로 출력
       64,         c8
```

자료형이 실수형이면 %f와 %e를 사용하여 출력할 수 있다. %f는 소수점 형태로 출력하는 서식이며, %e는 지수 형태로 출력하는 서식이다. 이때 디폴트로 소수점 이하 6자리를 출력하는데 이 소수점 이하 자리수는 .M과 같은 방식으로 M만큼 출력할 수 있다.

[표 D-2] 출력문에서 실수 출력 : 전체 크기와 소수점 이하 자리를 출력하는 방법

포맷	의미
{N.M}f	N칸에 맞게 소수점 이하 M자리를 소수점 형태로 출력
{N.M}e	N칸에 맞게 소수점 이하 M자리를 지수 형태로 출력

%e는 지수 표기법으로 123456789와 같은 수를 1.23456789 * 108과 같은 방식으로 표기할 수 있기 때문에, 출력시에는 1.23456789e+08 형식으로 표기한다. 이때 1.23456789는 가수부, 08은 지수부가 된다. 지수 표기시에도 소수점 이하 자리수를 .M과 같은 방식으로 M만큼 출력할 수 있다.

대화창 실습 : % 서식을 이용한 실수 출력

```
>>> print('%10f' % (3))    # 일반적인 실수 출력 : 소수점 아래 여섯 자리 출력
3.000000
>>> print('%10.2f' % (3))  # 실수 출력 : 소수점 아래 두 자리 출력
3.00
>>> print('%10.2f' % (3.1415926)) # 소수점 아래 두 자리 수 출력
3.14
>>> print('%10.3f' % (3.1415926)) # 소수점 아래 세 자리 수 출력
3.142
>>> print('%10.2e' % (123456789)) # 지수 표기법으로 수를 출력
1.23e+08
>>> print('%10.3e' % (123456789)) # 지수 표기법으로 수를 출력
1.235e+08
```

하지만 % 서식을 이용할 경우 문자열 다음에 format() 메소드를 이용하는 것보다 깔끔하지가 못하며 이해하기도 어렵다.

NOTE : format() 메소드와 % 서식지정자

format() 메소드는 파이썬 2에서 제공되었는데 이 방법을 사용하면 좀 더 복잡한 자료구조와 데이터 출력을 쉽게 할 수 있어서 %를 사용하는 것 보다 더 편리하고 안전하다. 예를 들어 다음과 같은 person 튜플이 있을 경우를 생각해 보자

```
person = ('Hong', 27, 179)
```

이 데이터를 출력하기 위하여 다음과 같이 % 표현식을 사용하게 되면 에러가 출력된다. 이 에러는 튜플 타입 person을 문자열 형식 출력인 %s로 표현할 수 없기 때문에 발생하는 에러이다.

```
>>> print('person = %s' % person)
...
TypeError: not all arguments converted during string formatting
```

반면 다음과 같이 format() 메소드를 사용하여 출력하게 되면 person 객체의 자체적인 출력 기능에 의해서 정상적인 출력이 이루어진다.


```
>>> print('person = {}'.format(person))
person = ('Hong', 27, 179)
```

format() 메소드의 상세한 기능

다음으로 format() 메소드를 이용하여 데이터를 출력할 때 필드 폭과 정렬을 조정하는 방법에 대해 상세하게 살펴보자. 이미 4장에서 다루었다시피 format() 메소드를 사용할 때는 {0:10.3f}와 같은 형식으로 10이라는 필드 폭을 지정해 줄 수 있는데 필드 폭 10에서 정렬은 디폴트로 오른쪽 칸을 기준으로 한다. 이 정렬은 '<', '>', '^' 기호를 이용하여 왼쪽 정렬, 오른쪽 정렬, 가운데 정렬을 지정할 수 있다. 그리고 이 기호 앞에 빈칸을 채울 문자열을 지정할 수 있다. 다음 대화창 실습은 출력필드의 폭이 30인 문자열의 출력 예제인데, 출력의 폭을 알기 위해 세로막대 '|'를 각각의 출력문의 마지막에 넣어주었다. 출력필드에서 빈칸은 디폴트로 공백문자이며 '*'를 빈칸 문자로 지정하여 출력할 수도 있다.

대화창 실습 : 왼쪽 정렬, 오른쪽 정렬, 가운데 정렬을 조절하는 출력

```
>>> print('{:<30}'.format('왼쪽 정렬 ')) # 30은 필드 폭, <은 왼쪽 정렬
왼쪽 정렬 |
>>> print('{:>30}'.format('왼쪽 정렬 ')) # 오른쪽 빈칸을 '*'로 채움
왼쪽 정렬 *****|
>>> print('{:>30}'.format('오른쪽 정렬')) # 30은 필드 폭, >은 오른쪽 정렬
오른쪽 정렬|
>>> print('{:*>30}'.format('오른쪽 정렬')) # 왼쪽 빈칸을 '*'로 채움
***** 오른쪽 정렬|
>>> print('{:^30}'.format('가운데 정렬 ')) # 30은 필드 폭, ^은 가운데 정렬
가운데 정렬 |
>>> print('{:^30}'.format('가운데 정렬 ')) # 빈 칸을 '*'로 채움
***** 가운데 정렬 *****|
```

다음으로 정수와 실수 출력시 부호를 삽입하는 방법에 대해 알아보자. {d}나 {f}와 같이 정수나 실수 출력을 하는 경우 디폴트로 부호 값을 넣지 않으며 음수인 경우에만 부호를 출력하며, 양수일 때는 부호를 출력하지 않는다. 이 때 +100, -100과 같이 항상 양과

음의 부호를 넣어서 출력을 하려면 어떻게 해야 할까? 이 경우에는 {:+d},{:+f}와 같이 +부호를 출력서식에 넣어주면 된다. 0을 출력하는 경우에도 +0과 같이 부호를 삽입하여 출력하는 것에 유의하도록 하자.

대화창 실습 : 부호 삽입을 가능하게 하는 포매팅

```
>>> print('{:d}, {:d}'.format(100, -100)) # 음수일 경우에만 부호 삽입
100, -100
>>> print('{:+d}, {:+d}'.format(100, -100)) # 항상 +, - 부호 삽입
+100, -100
>>> print('{:+f}, {:+f}'.format(3.14, -3.14)) # 항상 +, - 부호 삽입
+3.140000, -3.140000
>>> print('{:+d}'.format(0)) # 0일 때도 +, - 부호 삽입
+0
```

정수 값은 포맷터를 사용하여 10진수, 16진수, 8진수와 함께 2진수로도 출력이 가능하다. 정수를 10진수로 출력할 때는 d, 16진수로 출력할 때는 x, 8진수로 출력할 때는 o, 이진수로 출력할 때는 b라는 출력 포맷터를 사용한다. 이 정수 27을 출력하는 기능을 실습을 통해 알아보도록 하자. 출력 포맷터의 앞에 #을 삽입하면 0x, 0o, 0b와 같은 형식의 접두 문자를 삽입하여 각각 16진수, 8진수, 2진수임을 구분한다.

대화창 실습 : 10진수, 16진수, 8진수, 2진수 출력

```
>>> # 정수 27을 10진수, 16진수, 8진수, 2진수로 출력
>>> print('int: {0:d}, hex: {0:x}, oct: {0:o}, bin: {0:b}'.format(27))
int: 27, hex: 1b, oct: 33, bin: 11011
>>> print('int: {0:d}, hex: {0:#x}, oct: {0:#o}, bin: {0:#b}'.format(27))
int: 27, hex: 0x1b, oct: 0o33, bin: 0b11011
```

**NOTE : 8진수, 16진수, 2진수 출력**

정수 10을 8진수로 변환시킬 때는 `oct()` 함수를 사용하며, 16진수로 변환시킬 때는 `hex()`, 2진수로 변환시킬 때는 `bin()` 함수를 사용한다. 이때 정수는 `0o`, `0x`, `0b`라는 접두 문자를 통해 각각의 수를 구분한다.

```
>>> oct(10)
'0o12'
>>> hex(10)
'0xa'
>>> bin(10)
'0b1010'
```

출력 포매터는 이외에도 좀 더 다양한 기능이 있는데 `datetime`이라는 모듈의 연/월/일/시를 출력하는 훌륭한 기능도 제공하고 있다. `datetime` 모듈의 `datetime` 클래스는 연, 월, 일, 시를 지정하는 기능을 가지고 있는데 이 기능을 이용해서 지정한 날짜를 `{:%Y}`를 통해 출력하면 연도를 출력할 수 있다. 또한 `{:%m}`은 월을 출력할 수 있으며 이를 구분해서 출력하는 다양한 포매터도 제공하고 있음을 다음 실습 코드를 통해서 알아보도록 하자.

**대화창 실습 : datetime 모듈의 연/월/일/시 출력하는 방법**

```
>>> import datetime      # datetime 모듈을 import 함
>>> d = datetime.datetime(2019, 6, 12, 11, 10, 38)
>>> print('{:%Y-%m-%d %H:%M:%S}'.format(d))
2019-06-12 11:10:38
```

중괄호 내의 'Y', 'm', 'd', 'H', 'M', 'S' 등과 같은 문자는 각각 년(year), 월(month), 일(day), 시간(hour), 분(minute), 초(second)를 출력하는데 사용된다.

부록 E. 키보드 키의 이름

다음은 키보드 자판 키의 이름으로 위쪽은 한국어, 아래쪽은 영어로 읽는 방법이다. 키보드 자판의 정확한 명칭을 아는 것은 파이썬을 비롯한 많은 프로그래밍 개발에도 꼭 필요할 것이다.

기호	기호의 명칭
`	역따옴표 grave accent, back quote
~	물결표 tilde, wiggle
!	느낌표 exclamation mark, bang
@	골뱅이, 앳 사인 at sign, snail(달팽이)
#	해시, 샵 pound, number, sharp, hash
\$	달러 dollar
%	퍼센트기호 percent
^	캐럿, 꺾쇠기호 caret, circumflex, up arrow
&	앰퍼샌드 ampersand, and
*	별표, 스타 asterisk
\	역슬래시, 프로그래밍에서는 탈출문자라고도 함, 한글 키보드에서는 ₩로 표시되기도 함 back slash
:	: 콜론, 쌍점(colon)
;	; 세미콜론, 반쌍점(semi colon)
>, <	> : 크다(greater than) < : 작다(less than)
"	큰따옴표, 겹따옴표 quotation, quote

,	작은따옴표 apostrophe, single quote
.	마침표, 점 dot, period, full stop
?	물음표 question mark
(	왼쪽 괄호, 여는 괄호 open parenthesis
)	오른쪽 괄호, 닫는 괄호 close parenthesis
-	빼기표, 붙임표, 하이픈 minus, hyphen, dash
_	밑줄, 언더스코어, 언더바 underscore, underbar
+	더하기 plus
=	등호 equal
{, }	(여는, 닫는) 중괄호 (open, close) brace, curly brace
[,]	(여는, 닫는) 대괄호 (open, close) bracket
	수직선 pipe, bar, verti-bar
/	빗금, 슬래시, 나누기 slash, forward slash
fn	기능 키, 평션 키, 컴퓨터의 기능, 동작을 할당할 수 있음
Ctrl	컨트롤 키 - 다른 키와 조합하여 특수한 기능을 수행하는 키(^로 표시)
Alt	옵션 키라고도 하며 컨트롤 키와 마찬가지로 다른 키와 조합하여 기능수행
⌘	커맨드 키, 애플 키 - 맥 운영체제에서 다른 키와 조합하여 특수한 기능을 수행
 혹은 	윈도우 키 - 윈도 운영체제에서 다른 키와 조합하여 특수한 기능을 수행
Tab	탭 키 - 커서를 여러칸 움직이는 키
Esc	이스케이프 키, 탈출 키

Enter	엔터 키, macOS에서는 리턴 키
Shift	시프트 키 - 대/소문자를 입력하거나 한글 두벌식 키의 된소리입력시 사용
Caps Lock	캡스록 키 - 로마자 대/소문자를 바꾸는 키
Delete	삭제 키 delete key
Insert	삽입 키 insert key
PgUp, PgDn	페이지업, 페이지 다운 키 page up, page down
Home	홈 키 home